Chaining or Linear Probing in Practice: What's Better?

Samuel Nnawuihe-Echefu

February 2026

The following experiment done for *Data Structures and Algorithms I* at the University of Victoria, answering the mentioned conjecture. The paper delves into the problem by defining parameters of the experiment and comparing three popular language implementations to answer the question.

1 Conjecture

Consider the following conjecture.

For reasonable values of the load factor α, open addressing with linear probing (OALP) provides better performance than chaining.

2 Extracting Logic

In this section, I will define my methodology to answer this conjecture.

I will choose the following languages and implement a hash table to test load factors.

- Python
- Java
- C

The source code for the respective implementations will be provided in supplementary resources in the BrightSpace Dropbox.

3 Language Analysis

I will implement a hash table in each language using the same hash function and the Jenkins hashing algorithm in **Section 4.5** and built this function with minimal language-specific 'magic' to ensurean equal playing field.

3.1 Python

Python is a high-level, general-purpose language, and its core design philosophy focuses on code readability [6]. For the concerns of this experiment, Python is interpreted line by line, executing code after and changing memory addresses, which introduces extreme overhead during execution time, ultimately slowing down chaining.

3.2 Java

Java is compiled with Just-In-Time (JIT) compilation and is heavily optimized [1]. Java initially interprets code on first execution. On subsequent executions, Java finds parts of the program that are executed frequently and compiles them straight to machine code [3]. In Java, everything is a class, and classes themselves provide a lot of overhead. For chaining, a Linked List, Node, and Hash Table are compiled and ran which provides overhead and more movement with memory. For OALP, a list is placed side by side in memory, allowing for faster retrieval by the CPU.

3.3 C

C is compiled all at once into machine code, allowing for execution time to be incredibly fast. C by default, using the *GCC* compiler doesn't include any optimizations unless explicitly stated with the (-01) and (-02) flags [5]. C requires manual management and produces the same issues between chaining and OALP as arrays are placed inline in memory, allowing for fewer cache misses by the CPU. While implementing a linked list, node, and the table itself causes a lot of areas to leak memory, or slow down your program due to the movement of memory as well.

4 Hypothesis

When implementing a hash table, using open addressing with linear probing will provide better performance than chaining in practice.

4.1 Performance

Better performance in this paper will be defined by the total time taken to do each of the following actions:

- Resolve Collisions
- Find a given key
- Insert a given key
- Remove a given key

These performance metrics will be accounted for when testing.

4.2 Hash Function

The hash function to be used and replicated through the three languages is Jenkins bit Hash Function [2]. The procedure follows by looping over every character's or number's ASCII value of the key, applying it to the hash value, then shifting the hash value left by 10 and right by 6. After applying that transformation to the hash value, it shifts the hash value left by 3, then right by 11, then left by 15, and lastly masks the hash value by a 32-bit number. Below is the pseudo-code.

procedure $JenkinsHash(key)$ **do**
    $hash = 0$
    for all *char* in *key* **do**
        $hash = hash + ASCII(char)$
        shift *hash* 10 bits to the left
        shift *hash* 6 bits to the right
    end for
    shift *hash* 3 bits to the left
    shift *hash* 11 bits to the right
    shift *hash* 15 bits to the left
    return *hash*
end procedure

4.3 Hash Table Implementation

Each hash table will have the following operations.

$$Find(k) \tag{1}$$
$$Insert(k, v) \tag{2}$$
$$Remove(k) \tag{3}$$

The class implementations all have the following attributes:

- n : number of n values in the table
- α : load factor
- *table size*: the table size
- *table* : a list to contain the elements inserted.

4.4 Load Factor Values

For the experiments, I will test the group with $\alpha = 0.7$.

4.5 Table Size

The table size throughout each experiment will take into account 1009, $10,003$, $100,007$, and $1,000,007$ testing under different loads.

4.6 Measuring Findings

I will measure the outcomes across table size, load factor, and compare outputs across OALP and Chaining, with Python, Java, and C. Additionally, measure whether OALP ran faster than Chaining across Python, Java, and C. Each language will run insert, find, and remove operations proportional to the hash table's initial size, both randomly and sequentially.

4.6.1 Why insert randomly?

In practice, it is incredibly unlikely to predict that the keys being inserted into the table are sequential (or sorted). Randomly inserting keys can mimic real-world behavior.

4.6.2 Why insert sequentially?

There may be a use case for a hash table with the different collision schemes, where the keys are sequential (or sorted). Sequentially inserting keys covers this use case and measures the behavior of the hash table in such a scenario.

4.7 Compiling and running code

The following is how I compiled and ran each file.

```
# Running Python code
> time python3 main.py

# Running Java Code
> javac HashTableOALP.java
> time java HashTableOALP

> javac HashTableChaining.java
> time java HashTableChaining

# Running C Code
> gcc ...
> time ./main
```

5 Python Table Findings

5.1 Random Values

5.1.1 $\alpha = 0.7$

Table Size (m)	OALP (s)	Chaining (s)
1009	0.054	0.051
10,003	0.134	0.141
100,007	1.091	1.136
1,000,007	11.106	15.056

5.2 Sequential

5.2.1 $\alpha = 0.7$

Table Size (m)	OALP (s)	Chaining (s)
1009	0.198	0.024
10 003	5.0206	0.0847
100 007	192.092	1.065

6 Java Table Findings

6.1 Random Values

6.1.1 $\alpha = 0.7$

Table Size (m)	OALP (s)	Chaining (s)
1009	0.070	1.031
10,003	0.088	1.321
100,007	0.686	1.383
1,000,007	7.949	2.187

6.2 Sequential

6.2.1 $\alpha = 0.7$

Table Size (m)	OALP (s)	Chaining (s)
1009	0.245	0.045
10 003	6.037	0.155
100 007	180.007	0.883

7 C Table Findings

7.1 Random Values

7.1.1 $\alpha = 0.7$

Table Size	OALP (s)	Chaining (s)
1009	0.003	0.003
10,003	0.012	0.014
100,007	0.137	0.161
1,000,007	2.139	2.765

7.2 Sequential

7.2.1 $\alpha = 0.7$

Table Size	OALP (s)	Chaining (s)
1009	0.003	0.003
10,003	0.020	0.008
100,007	0.261	0.098
1,000,007	14.429	1.679

8 Analysis

$$Speedup_{tablesize} = \left(\sum \frac{Chaining_{tablesize}}{OALP_{tablesize}} \right) \div 3$$

The above formula computes the sum of the ratio by comparing the time taken by chaining against OALP for a given table size. By the hypothesis, if the average of speedup is < 1 than take the inverse of the average and multiply it by -1 to get the average speedup (or slowdown):

$$\frac{-1}{Speedup}$$

8.1 Random

8.1.1 Load Factor $\alpha = 0.7$

Table Size	Sum of Speedup	OALP Average (Sum of Speedup/3)
1009	15.973	5.324x
10,003	17.230	5.743x
100,007	4.232	1.411x
1,000,007	2.923	-1.027x

8.2 Sequential

8.2.1 Load Factor $\alpha = 0.7$

Table Size	Sum of Speedup	OALP Average (Sum of Speedup/3)
1009	0.972	-3.088x
10,003	0.1475	-6.779x
100,007	0.386	-2.591x

9 Conclusion

It is not the case that open addressing is faster in practice; open addressing with linear probing will perform better than chaining for reasonable values of a load factor.

In the speedup differences, for a load factor of 0.7, open addressing shows a performance dip as the table size grows. For a table size of $1,000,007$ when random keys are used, the performance decreases for OALP. In sequential values, OALP only performs at a decrease in practice. The reason why the values for sequential don't account for $1,000,007$ is that Python and Java's speedup for OALP is actually $95.698\times$ slower than chaining for $100,007$. By heuristics, it would be redundant to test a bigger input if the previous input is less than the bigger one, expecting a faster time.

The reason why it matters as input grows, Amazon manages over 350 trillion objects and 100 million requests per day [4]. If Amazon uses open addressing with linear probing, it would increasingly take more time to process such an input. Not all situations will scale up to Amazon, but if an application or service needs to handle 6 keys per user ('username', 'password', 'email', ...) and have $20,000$ users, it compounds to $120,000$ keys inserted. Per the table data, you'll begin to see a significant drop in performance due to the collision scheme, given an initial table size of $100,000$.

References

[1] Sakshee Agrawal. *Understanding Just-In-Time (JIT) Compilation in Java*. URL: https://medium.com/@sakshee_agrawal/understanding-just-in-time-jit-compilation-in-java-ae2a6b9fa931. (accessed: Feb 22, 2026).

[2] Mojo Auth. *Jenkins Hash Function In Python*. URL: https://mojoauth.com/hashing/jenkins-hash-function-in-python#introduction-to-jenkins-hash-function. (accessed: Feb 10, 2026).

[3] Gunavaran Brihadiswaran. *A Performance Comparison Between C, Java, and Python*. URL: https://medium.com/swlh/a-performance-comparison-between-c-java-and-python-df3890545f6d. (accessed: Feb 22, 2026).

[4] bytebytego. *How Amazon S3 Stores 350 Trillion Objects with 11 Nines of Durability*. URL: https://blog.bytebytego.com/p/how-amazon-s3-stores-350-trillion. (accessed: Feb 22, 2026).

[5] GCC. *Optimize Options (Using the GNU Compiler Collection (GCC))*. URL: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html. (accessed: Feb 20, 2026).

[6] Python. *What is Python? Executive Summary — Python.org*. URL: https://www.python.org/doc/essays/blurb/. (accessed: Feb 10, 2026).